BayesianEstimationTool Manual

BayesianEstimationTool (BET) helps you learn from a group of related individuals and make predictions for someone new. An **individual** (also called an *entry* in some API descriptions) might be a patient, an experiment, or a simulated system. This manual uses individual for all three; the examples happen to use people. You provide a model of how the system works and the measurements you have collected; BET estimates its unknown parameters and helps you test its predictions.

Imagine measuring how quickly tumors grow in several patients. The same growth equation might describe every patient, but each patient's growth rate can differ. If you estimate each rate separately, a patient with only one or two measurements gives you little information. BET instead learns from the whole group: what growth rates are typical, how much they vary, and what the measurements tell us about each patient.

You can predict a new patient's trajectory *before seeing any measurements*, using what BET learned from the group. *After seeing a few measurements*, you can update that prediction using information from the patient. BET also helps you measure whether those predictions were accurate and how uncertain they are.

This approach is called **hierarchical Bayesian inference**. You do not need to master the terminology to use the basic workflow. Throughout the manual, **population** means the group whose shared patterns BET learns; it does not require a medical or demographic population.

Installation

BET requires Python 3.10 or newer. **After the first release is published**, install the latest version from PyPI:

```
python -m pip install BayesianEstimationTool
```

To install the specific first release:

```
python -m pip install BayesianEstimationTool==1.0.0
```

Alternatively, install directly from the tagged GitHub source:

```
python -m pip install "git+https://github.com/rafaelrbravo/BayesianEstimationTool.git@v1.0.0"
```

You can also download a wheel (`.whl`) or source archive (`.tar.gz`) from GitHub Releases and install it with `python -m pip install PATH_TO_DOWNLOADED_FILE`. These release assets become available after publishing.

Verify installation:

```
python -c "import BayesianEstimationTool as bet; print(bet.Estimator)"
```

For editable development from a source checkout, run `python -m pip install -e .` in the repository root. The conda recipe in `conda-recipe/meta.yaml` is for local builds; BET is not yet published on conda-forge.

The workflow at a glance

1. **Organize observations.** Put each individual's measurements and inputs into arrays.
2. **Describe the system.** Write a function that predicts what happens when its parameters are known.
3. **Describe what is unknown.** Specify which parameters are shared, which vary by individual, and how measurements differ from ideal predictions.
4. **Learn from the group.** Use `Train` to estimate the population and individual parameters.
5. **Predict for new individuals.** Use `Test` with or without their available measurements.
6. **Check the predictions.** Use scoring functions to compare predictions with observations held back from fitting.

Core components

Component	What it does
`data`	Holds measurements and inputs for each individual
`ModelFn`	Predicts what happens for one individual
`GlobalPriorFn`	Describes plausible values for unknown shared parameters before fitting
`localZNames`	Names the ways individuals can differ from one another
`TrainLikelihoodFn`	Describes how training observations relate to model predictions
`TestLikelihoodFn`	Describes how available observations from new individuals update their local latent z-scores
`Method` (`Nuts` / `Svi`)	Chooses how BET estimates unknown parameters from observations
`Estimator`	Connects your model, data, and inference results
`Train` / `Test`	Learns from the training group / predicts for new individuals
`ScoreFn`	Measures how well a prediction matches observations

Part 1: Core concepts

What BET learns

A model is a rule for making predictions. For example, a growth equation uses a growth rate and a time to predict a size. BET does not choose that equation for you; you write it in `ModelFn`. BET helps determine which parameter values are consistent with your measurements.

Some parameters describe the group; others describe individuals. A population might have a typical growth rate, while each patient's tumor grows somewhat faster or slower. BET learns about both the shared pattern and the individual differences. In the API, shared unknown parameters are called **global parameters**. Individual differences are represented by **local latent z-scores** (`localZs`): standardized, unobserved values that describe how each individual differs from the group. Your model combines these z-scores with global parameters to obtain meaningful individual parameters, such as each patient's tumor growth rate.

Measurements are imperfect. Even a good model will not match every observation exactly. The **likelihood** describes how observations can differ from model predictions, including measurement noise. A **prior** describes plausible parameter values before

those observations are used. BET combines the prior information with the measurements to estimate what values are plausible afterward; these updated estimates form the **posterior**.

Learning and predicting are different steps. During `Train`, BET uses the selected individuals to learn the global parameters and each training individual's local latent z-scores. During `Test`, it predicts for individuals not included in training. Without their measurements, BET draws possible local latent z-scores from the learned population. With some measurements and a test likelihood, BET updates those z-scores for each new individual. The current `Test` workflow requires at least one named local z-score (`localZNames`); a model with only global parameters can be trained but cannot use `Test` directly. To assess prediction quality fairly, keep the observations you want to predict out of the fitting step.

BET tracks uncertainty rather than returning only one answer. Different plausible parameter values can produce different predictions. Samples from the fitted distribution let you examine that variation. Predictions of the underlying model are not automatically noisy future measurements; the observation model describes that additional variation.

Designing a BET model

To implement this workflow:

1. Prepare `data` as a dictionary of arrays. Every array's first axis identifies the individual; later axes can represent times, features, or positions.
2. Write `ModelFn(globalParams, localZs, data)` to predict outputs for *one individual*. Return a dictionary of named outputs. Keep this function deterministic: do not sample random variables inside it.
3. If shared parameters are unknown, define `GlobalPriorFn(dataShapes)` to give their prior distributions. Otherwise, omit it.
4. Use `localZNames` for the individual differences you want BET to estimate. Inside `ModelFn`, transform these standardized effects into meaningful parameters such as growth rates.
5. Define `TrainLikelihoodFn` to connect observations to predictions using distributions. If measurements are missing, mask them explicitly.
6. Choose training individuals and an inference method, `Nuts(...)` or `Svi(...)`. Call `Train` and check whether the fitted model describes the data reasonably.
7. Choose held-out individuals. Call `Test` for predictions from the population alone, or provide a test likelihood and inference method to update local z-scores using measurements available at prediction time.
8. Define a scoring function and call `ScoreTrainSamples` or `ScoreTestSamples` to evaluate predictions. Keep track of which dimensions represent samples and which represent individuals.

Automatic vectorization

Write `ModelFn`, `TrainLikelihoodFn`, `TestLikelihoodFn`, and `ScoreFn` for one individual at a time. BET applies them across individuals and parameter samples using JAX vectorization, so you do not need to write those loops yourself. These functions should use JAX-compatible operations and avoid side effects. Vectorization enables efficient batched computation, although actual parallel execution depends on the operations and hardware.

Part 2 combines these steps in a tumor growth example. Later sections explain correlated local z-scores and inference-method configuration.

Part 2: Worked example

Predicting growth for someone new

Suppose we measure tumor size eight times for each of ten patients. We want to learn how tumor growth rates differ across patients, then predict the last five measurements for four patients excluded from training. We will first predict using only what we learned from the other six patients, then see whether the first three measurements from each new patient improve the predictions.

We generate synthetic data so you can follow the workflow without downloading a dataset. The growth equation is deliberately simple, but the workflow also applies to more detailed scientific models.

Create example measurements and a growth model

```
import jax.numpy as jnp
from jax import random
import numpyro.distributions as dist
from BayesianEstimationTool import Estimator, Svi

n_people = 10
times = jnp.linspace(0.0, 7.0, 8)
key_g, key_noise = random.split(random.PRNGKey(42))
true_g = 0.15 + 0.025 * random.normal(key_g, (n_people,))
true_y = jnp.exp(true_g[:, None] * times[None, :])
y = true_y + 0.05 * random.normal(key_noise, true_y.shape)
data = {"time": jnp.tile(times, (n_people, 1)), "y": y}

def global_prior(shapes):
    return {
        "g_mean": dist.Normal(0.15, 0.10),
        "g_sd": dist.HalfNormal(0.10),
        "noise": dist.HalfNormal(0.10),
    }

def model_fn(global_params, local_zs, data):
    g = global_params["g_mean"] + global_params["g_sd"] * local_zs["g"]
    return {"prediction": jnp.exp(g * data["time"])}

def train_likelihood(global_params, local_zs, data, model_out):
    return {"y": (data["y"], dist.Normal(model_out["prediction"], global_params["noise"]))}
```

The model has three unknown population-level quantities. `g_mean` describes a typical growth rate; `g_sd` describes how much rates differ between people; and `noise` describes random variation in measurements. The individual value `local_zs["g"]` says where one person falls relative to the typical rate. The model converts it to that person's growth rate using `g_mean + g_sd * z`.

`global_prior` describes plausible starting ranges for the three unknown quantities. `train_likelihood` says that each measurement is expected to be near the model's prediction, with variation controlled by `noise`. The data generator uses known values only to create this synthetic example; BET does not see those values when fitting.

You can run the Python blocks in this worked example in order, in one Python session. Part 5 reuses the resulting `est`, `before`, and `after` variables rather than repeating the fits. The other code blocks in the manual illustrate separate features and are not all intended to run as one script.

Train on six individuals

```
est = Estimator(data, model_fn, localZNames=["g"],
                GlobalPriorFn=global_prior, rngSeed=123)
train_ids = list(range(6))
test_ids = list(range(6, 10))

est.Train(train_ids, train_likelihood,
          Svi(numSteps=1500, numSamples=300, progressBar=False))
```

The first six people form the *training group* used to fit the model. BET uses their measurements to estimate typical growth, between-person variation, and measurement noise. The other four form the *test group*: their measurements are excluded from training.

Here `Svi` is the inference method that estimates the unknown quantities. These settings are illustrative, not a guarantee of accurate inference. For research, check the fitted predictions and sensitivity to modeling choices.

Predict using the group alone

To see how well our predictions work, we calculate the average absolute difference between predictions and measurements for the *last five time points* (indices 3 through 7). These measurements are available for evaluation, but we must not use them to personalize the predictions.

```
def future_mae(model_out, data):
    return jnp.mean(jnp.abs(model_out["prediction"][3:] - data["y"][3:]))

est.Test(test_ids, nGlobalSamples=20, nLocalZSamples=100)
before = est.ScoreTestSamples(future_mae)
print("Population-only mean future MAE:", float(jnp.mean(before)))
assert before.shape == (20, 100, 4)
```

BET considers 20 plausible versions of the learned population and 100 possible growth-rate differences for each one. We did not supply a `TestLikelihoodFn` to `Test`, so BET does not use any measurements from the test patients to estimate their local latent z-scores. Instead, it samples those z-scores from the learned population, producing **population-only predictions**.

The score array has shape `[20, 100, 4]`: global-parameter draw, local z-score draw, and test person. Each number measures a possible prediction's error.

Personalize predictions using three measurements

Now suppose we have received the first three measurements for each test person. We can use those observations to estimate their growth rate more specifically. A **mask** marks which measurements BET is allowed to use for that update. The last five remain hidden from inference, even though we retain them to check the predictions afterward.

```
def test_likelihood(global_params, local_zs, data, model_out):
    observed = jnp.arange(data["y"].shape[0]) < 3
    prediction = model_out["prediction"]
    safe_y = jnp.where(observed, data["y"], prediction)
    return {
        "y": (safe_y, dist.Normal(prediction, global_params["noise"]).mask(observed))
    }
```

```
est.Test(test_ids, TestLikelihoodFn=test_likelihood,
         Method=Svi(numSteps=800, numSamples=100, progressBar=False),
         nGlobalSamples=20)
after = est.ScoreTestSamples(future_mae)
print("Conditioned mean future MAE:", float(jnp.mean(after)))
assert after.shape == (20, 100, 4)
```

BET updates each test person's growth-rate estimate using their first three measurements and the population information learned earlier. We still score only the last five measurements, which BET did not use for the update. Calling `Test` again replaces the previous test state inside `est`, but the earlier `before` scores remain in their own variable. *Runtime tip:* The later personalized-prediction step repeats inference for each of the 20 sampled global parameter states, so it may take time. For a quicker run, reduce `nGlobalSamples` in both `Test` calls to the same value and update the corresponding shape assertions; you can also reduce the SVI step counts.

What did the extra measurements buy us?

Both `before` and `after` have shape `[20, 100, 4]`. Their means summarize prediction errors across plausible global parameters, local z-scores, and test patients. Because we measure mean absolute error, lower values mean more accurate predictions of the held-out measurements. If the `after` mean is lower, using the first three measurements improved predictions of the remaining five on average. With only ten synthetic patients, an improvement is not guaranteed.

Inspecting the full inference results

Scoring reduces predictions to an error measure, but BET also lets you examine the underlying posterior samples directly. **`est.Save(None)` returns a dictionary of results in memory instead of writing a file.** It includes sampled global parameters (`"trainGlobals"` and `"testGlobals"`) and, when retained, individual local z-scores (`"trainLocalZs"` and `"testLocalZs"`), along with information needed to save or restore the estimator. This lets you inspect parameter uncertainty and differences between patients, not just prediction errors.

To inspect predicted trajectories as well, run `Test` with `saveModelOut=True` and request those outputs when retrieving the results:

```
samples = est.Save(None, saveTestModelOut=True)
trajectories = samples["testModelOut"]["prediction"]
```

The matching parameter draws are in `samples["testGlobals"]` and `samples["testLocalZs"]`. Summarize the trajectories across posterior draw axes to obtain uncertainty intervals. These intervals describe latent mean trajectories; they do not automatically include new measurement noise. For stronger evaluation, repeat across random seeds and vary the number of early observations.

The *Train* → *Test* → *Score* workflow also applies to more complex models, including differential-equation models.

The assertions check score dimensions, not predictive accuracy; the printed MAE values are not expected to match fixed numbers. Part 7 explains how to evaluate predictions without data leakage.

Part 3: Learning correlations between local z-scores

The worked example has one individual-specific parameter: growth rate. BET can represent several individual differences and learn whether they tend to occur together.

BET represents individual-to-individual differences using **local z-scores** on a standardized scale. For example, `z=0` gives the model's central growth rate; positive or negative values change it according to the transformation you define. With `rate = exp(log_rate_mean + log_rate_sd * z)`, `log_rate_mean` sets the center on the log scale and `log_rate_sd` sets its spread. Before conditioning on individual observations, each z-score has a standard-normal marginal distribution.

Extend the tumor-growth example by adding treatment. Each patient now has two parameters: a **growth rate**, describing how quickly the tumor grows without treatment, and a **treatment-response parameter**, describing how strongly treatment slows that growth. Both may vary between patients. For instance, patients whose tumors grow faster might also tend to respond more strongly to treatment, or the relationship might be reversed. Assuming the parameters vary independently would miss that population-level relationship.

For example, this schematic treatment model transforms each patient's local z-scores into positive growth and response parameters. It illustrates the transformation, not a validated description of tumor dynamics:

```
def model_fn(global_params, local_zs, data):
    growth = jnp.exp(global_params["log_growth_mean"] + global_params["log_growth_sd"] *
local_zs["growth"])
    response = jnp.exp(global_params["log_response_mean"] + global_params["log_response_sd"] *
local_zs["response"])
    net_rate = growth - response * data["treatment"]
    return {"prediction": data["baseline"] * jnp.exp(net_rate * data["time"])}
```

You define the global means, scales, and transformations in the model; BET does not create them automatically.

Set `localZNames=["growth", "response"]` to represent both sources of patient-to-patient variation as standardized local z-scores. BET can learn their **correlation** from the training patients. A positive correlation means patients with above-average growth z-scores tend also to have above-average response z-scores; a negative correlation means higher growth z-scores tend to accompany lower response z-scores. These are correlations between the standardized effects, not necessarily between the parameters after arbitrary transformations.

BET represents this relationship using a **correlation matrix**. Internally, it uses a **Cholesky factor** of that matrix to generate correlated local z-scores. A correlation matrix specifies how pairs of effects vary together; its Cholesky factor is a triangular matrix that can transform independent standard-normal draws into correlated ones. BET places an **LKJ prior** on the correlation structure, with `lkjConcentration` controlling how strongly that prior favors correlations near zero. The inferred relationship can improve predictions for a new patient: measurements that inform their tumor growth rate can also inform their likely treatment response, even if response measurements are limited.

Choosing `lkjConcentration`. The LKJ prior changes with this setting:

- `lkjConcentration=1`: All valid correlation matrices have equal prior density. With more than two local z-scores, this does not mean that each pairwise correlation is uniformly distributed.
- `lkjConcentration=2` *(default)*: Mildly favors weaker correlations. This is a reasonable starting point for most models.
- `lkjConcentration=4`: Favors weaker correlations more strongly; useful when large correlations seem unlikely before seeing the data.
- `lkjConcentration<1`: Favors more extreme correlation structures; generally avoid unless strong correlations are expected.

When data are limited, try more than one concentration to check how much the inferred correlations depend on the prior.

Part 4: Building blocks

Parts 2 and 3 showed how to train a model, make predictions, and represent correlated differences between individuals. This part serves as a reference for the functions and arguments used in that workflow: how to configure an `Estimator`, run `Train` and `Test`, score predictions, and define the model, priors, and likelihoods they require.

Core workflow: Estimator, Train, Test, and Score

`Estimator(data, ModelFn, localZNames=None, GlobalPriorFn=None, lkjConcentration=2.0, rngSeed=None)` sets up the model:

- `data`: A dictionary of arrays, each with individuals along the first axis. All arrays must have the same number of individuals; remaining axes can represent times, features, or spatial coordinates. For example, measurements from 10 patients at 8 times give `data["y"]` shape `(10, 8)`; the corresponding one-patient shape, `dataShapes["y"]`, is `(8,)`.
- `ModelFn(globalParams, localZs, data)`: Receives dictionaries of global parameters, one individual's local z-scores, and that individual's data; returns a dictionary of model outputs. Use JAX-compatible operations without Python side effects or converting traced values to NumPy.
- `localZNames`: Names of the local latent z-scores to infer; omit when there are no individual-specific latent variables.
- `GlobalPriorFn(dataShapes)`: Returns a dictionary mapping shared global parameter names to NumPyro prior distributions. `dataShapes` contains each array's shape for one individual, without the leading individual axis. Optional when there are no global parameters to infer.
- `lkjConcentration`: Controls the correlation prior for multiple local z-scores (default `2.0`); see Part 3.
- `rngSeed`: Optional random seed for reproducible sampling.

`est.Train(trainDataIdxs, TrainLikelihoodFn, Method, printSummary=False, rngSeed=None, saveLocalZs=True, saveModelOut=False)` learns from the training individuals:

- `trainDataIdxs`: Indices of the individuals used for training.
- `TrainLikelihoodFn(globalParams, localZs, data, modelOut)`: Returns a dictionary mapping observation site names to `(observed_value, distribution)` tuples. Each site can contain scalar or vector observations; observation, distribution, and mask shapes must agree.
- `Method`: Configured inference method, such as `Nuts(...)` or `Svi(...)`, or a supported Method class that can reuse saved configuration.
- `printSummary`: Whether to print an inference summary.
- `rngSeed`: Optional seed for this training run.
- `saveLocalZs`: Whether to retain training local z-score samples (default `True`). For later training scores, retain either these draws or model outputs.
- `saveModelOut`: Whether to retain model outputs for the training samples.

`est.Test(testDataIdxs, TestLikelihoodFn=None, Method=None, nGlobalSamples=None, nLocalZSamples=2000, printSummary=False, rngSeed=None, saveModelOut=False)` makes predictions for test individuals:

- `testDataIdxs`: Indices of the individuals to predict.
- `TestLikelihoodFn(globalParams, localZs, data, modelOut)`: Optional function with the same return format as `TrainLikelihoodFn`, using available test observations to update local z-scores. Omit for population-only predictions.
- `Method`: Inference method for that update, when a test likelihood is supplied; it can reuse saved configuration.
- `nGlobalSamples`: Number of sampled global parameter states used for testing; `None` uses the posterior median global state; a positive integer samples training posterior states.

- `nLocalZSamples`: Number of local z-score draws per global state for population-only predictions.
- `printSummary`: Whether to print an inference summary.
- `rngSeed`: Optional seed for this test run.
- `saveModelOut`: Whether to retain test model outputs.

`est.ScoreTrainSamples(ScoreFn, nSamples=None, rngSeed=None)` and `est.ScoreTestSamples(ScoreFn, nSamples=None, rngSeed=None)` evaluate predictions:

- `ScoreFn(modelOut, data)`: Compares one individual's model output with their data and returns a scalar or array. Training scores have axes (draw, training individual, score dimensions); test scores have axes (global draw, local draw, test individual, score dimensions).
- `nSamples`: Optional number of posterior draws to score; `None` uses all available draws.
- `rngSeed`: Optional seed when selecting draws.

Additional API details

Models without local z-scores. `Estimator` and `Train` support models without local z-scores, but `Test` currently requires at least one (`Test() requires at least one local Z`). Consequently, `ScoreTestSamples` is unavailable through that workflow.

Saving and loading. `est.Save(fileName, saveTrainLocalZs=True, saveTestLocalZs=True, saveTrainModelOut=False, saveTestModelOut=False, saveAllData=False)` saves the estimator state to a compressed file. With `fileName=None`, it returns the results as a dictionary instead. `Estimator.Load(fileName, ModelFn, GlobalPriorFn=None, data=None)` reconstructs the estimator; supply `data` unless it was saved. With `ModelFn=None`, it returns the saved record without reconstructing an estimator. See Part 6 for examples and persistence options.

Custom methods. `ValidateMethod(method)` checks whether a custom inference method implements BET's required interface. See **Advanced: Adding a custom inference method** at the end of the manual for implementation details.

Part 5: How BET learns from data

After you define the model and observations, BET estimates plausible parameter values using one of two built-in inference methods. Both produce parameter samples, but their algorithms and computational costs differ.

Choosing between NUTS and SVI

NUTS explores plausible parameter values by drawing samples through a Markov chain. It is often useful when the shape and spread of uncertainty matter. `Nuts(numWarmup=2000, numSamples=2000, numChains=4, acceptProb=0.95, denseMass=False, medianSamples=50)` uses NumPyro's NUTS MCMC kernel with median-based initialization. `numWarmup` is the adaptation period; `numSamples` is the retained number of draws **per chain**; `numChains` controls chain count; `acceptProb` is the target acceptance probability; `denseMass` enables dense mass-matrix adaptation; and `medianSamples` controls the initialization strategy. BET combines the chains into one leading draw axis for downstream scoring, so a run with four chains and 2,000 retained draws per chain provides up to 8,000 training posterior draws before any subsampling. Its reported summary includes effective sample size and R-hat. Inspect divergences, mixing, and posterior predictive adequacy rather than treating a successful run as validation.

SVI takes a different approach: it fits a simpler probability distribution to approximate the range of plausible parameter values. It can be faster, although the approximation may miss some uncertainty. `Svi(numSteps=10000, numSamples=2000, stepSize=0.01, numParticles=1, progressBar=True, medianSamples=50, multivariate=False)` optimizes a variational approximation with Adam and an ELBO objective. `numSteps` is the number of optimization updates; `numSamples` controls approximate posterior draws generated after optimization; `stepSize` is Adam's learning rate; `numParticles` controls Monte

Carlo particles for the ELBO; and `medianSamples` controls initialization. `multivariate=False` uses `AutoNormal`; `True` uses `AutoMultivariateNormal`, which can represent posterior covariance but may be expensive in large hierarchical models.

Method selection. NUTS is often preferable when reliable posterior uncertainty is central and runtime is manageable. SVI can be faster for large problems, but an apparently stable loss curve does not establish calibrated uncertainty. Compare posterior predictive behavior and uncertainty against NUTS or simulated ground truth when feasible. You can pass a configured method, such as `Svi(numSteps=5000)`, directly to `Train` or `Test`. After loading a saved estimator, passing the method class (such as `Svi`) lets BET reuse its saved settings, if available.

Custom methods. Developers can integrate algorithms beyond NUTS and SVI through BET's Method interface. See **Advanced: Adding a custom inference method** and `ValidateMethod`.

Part 6: Saving your work

Once you have trained a model, you may want to return to it later without running inference again. BET can save the results of training and testing, along with enough configuration to reconstruct an estimator.

Saving and loading

`Save` preserves the fitted population posterior, training/test configuration, selected entry indices, and RNG state in a compressed `.npz` file. You can also choose whether to retain individual local-effect samples, model outputs, and the original data.

```
est.Save("fit.npz", saveAllData=True)
restored = Estimator.Load("fit.npz", ModelFn=model_fn, GlobalPriorFn=global_prior)
```

The full signature is:

```
Save(fileName, saveTrainLocalZs=True, saveTestLocalZs=True,
     saveTrainModelOut=False, saveTestModelOut=False, saveAllData=False)
```

Option	What it controls
`fileName`	Destination for the compressed `.npz` file. If `None`, return an in-memory nested record instead of writing a file.
`saveTrainLocalZs`	Include training individuals' posterior local-effect samples. Enabled by default.
`saveTestLocalZs`	Include the latest test individuals' local-effect samples and their associated sampled global/Cholesky states. Enabled by default.
`saveTrainModelOut`	Include previously retained training model outputs, if available. Disabled by default.
`saveTestModelOut`	Include previously retained test model outputs, if available. Disabled by default.
`saveAllData`	Include the original input data, allowing loading without supplying it separately. Disabled by default.

Population posterior samples and saved Train/Test configuration are retained independently of these options. Omitting local samples or model outputs reduces file size but may limit subsequent scoring or require recomputation. In particular, to score a loaded training fit with local z-scores, retain training local draws or previously saved model outputs; to score loaded test

predictions, retain test local draws or previously saved test model outputs. The latter outputs must have been requested during `Test` and included during `Save`. The model-output options only save outputs already retained by `Train(..., saveModelOut=True)` or `Test(..., saveModelOut=True)`; they do not generate missing outputs at save time.

To restore an estimator:

```
restored = Estimator.Load("fit.npz", ModelFn=model_fn,
                          GlobalPriorFn=global_prior, data=data)
```

`Load(fileName, ModelFn, GlobalPriorFn=None, data=None)` reconstructs the saved estimator, including available posterior samples and configuration. Omit `data` if the file was saved with `saveAllData=True`; otherwise supply the original data. You must supply the model function and, when used, the global-prior function again because Python functions are not serialized. BET warns when their names differ from the saved names, but a matching name does not guarantee identical behavior.

For inspection without reconstructing an estimator, call `Estimator.Load("fit.npz", ModelFn=None)` to obtain the saved record as a dictionary. Saved Method names and constructor arguments support reconstruction of inference configuration; they do not serialize live NUTS or SVI engine objects.

Part 7: Working with incomplete data and numerical models

Individuals often have different numbers of measurements, and some measurements may be missing. BET works with arrays, so these differences need to be represented explicitly without pretending missing values were observed.

Different numbers of measurements

Use masks when entries have different observation counts, measurements are missing, or testing should reveal only part of a trajectory. A mask distinguishes *positions stored in an array* from *observations contributing to inference*. Held-out measurements can remain in the data for scoring while being excluded from the test likelihood.

Pad each time series to a common length `T`. Store `time`, `y`, and a boolean `valid` mask as `[N, T]` arrays; padded positions are placeholders, not observations. Choose finite placeholder measurements that are safe for any transformations in your model; repeating the last valid measurement is often useful. Choose padded times separately to satisfy your solver's requirements: some solvers reject repeated times and need a strictly increasing grid. Exclude all padded positions from the likelihood.

```
data = {
    "time": jnp.array([[0., 1., 2.], [0., 3., 4.]]),
    "y": jnp.array([[1.0, 1.2, 1.4], [0.9, 1.1, 1.1]]),
    "valid": jnp.array([[True, True, True], [True, True, False]]),
}
```

Use a masked NumPyro distribution and replace invalid observed values with finite, safe values before log-probability evaluation:

```
def likelihood(global_params, local_zs, data, model_out):
    valid = data["valid"]
    pred = model_out["prediction"]
    safe_y = jnp.where(valid, data["y"], pred)
    return {"y": (safe_y, dist.Normal(pred, 0.2).mask(valid))}
```

The model must produce finite predictions even at padded positions. Masking cannot make an intermediate `NaN` or infinity safe; multiplying a divergent log-probability by zero does not guarantee a finite gradient. Here the second individual's final measurement is a placeholder, and its padded time advances from 3 to 4 to keep the time array increasing. The validity mask excludes that position from inference. A real ODE model still needs a time grid compatible with its solver and finite predictions at padded positions.

For partial-observation testing, use a separate mask such as `test_observed = valid & (time <= cutoff)`. Apply it in the test likelihood, but retain the original validity mask for scoring held-out observations. Ensure hidden measurements cannot enter the model through inputs or preprocessing.

Models defined by differential equations

Some scientific models specify *rates of change* rather than values directly. A differential equation solver converts those rates into a predicted trajectory. `ModelFn` can call a JAX-compatible ODE solver such as `jax.experimental.ode.odeint`. Use JAX arrays and differentiable operations, and return outputs at observation times. In the workflows tested for this project, `odeint` has worked more reliably with BET's batching than Diffrax. Compatibility is model-dependent, so verify gradients and numerical behavior. Diffrax is an optional external dependency, not part of BET's installation requirements.

```
from jax.experimental.ode import odeint

def rhs(state, time, rate):
    return rate * state

def model_fn(global_params, local_zs, data):
    rate = jnp.exp(global_params["log_rate"] + local_zs["rate"])
    times = data["time"]  # sorted, finite, compatible with odeint
    trajectory = odeint(rhs, data["initial"], times, rate)
    return {"prediction": trajectory}
```

Padded time grids must satisfy the solver's ordering requirements. One approach is to solve on a shared regular grid and interpolate to each individual's observation times. Check for duplicate or unsorted padded times.

Part 8: Evaluating predictions without data leakage

A model can reproduce its training data yet predict poorly for new individuals. Part 2 demonstrated scoring; here the focus is on designing an evaluation that measures genuine predictive performance.

Separate fitting, personalization, and evaluation

Keep test individuals out of training. For each test individual, distinguish measurements available **at prediction time** from later outcomes reserved for evaluation. In the worked example, the first three measurements can update the individual's local z-scores, while the final five are held out for scoring. Neither the model inputs nor the test likelihood should expose those future outcomes during prediction.

Training scores answer a different question: how well does the fitted model describe individuals it learned from? They can reveal poor fit but are not an unbiased estimate of performance on new individuals. Population-only test scores measure prediction without individual observations; personalized test scores measure prediction after using the permitted early observations. Compare the latter two against the **same held-out outcomes** to assess the value of personalization.

Interpret scores at the right level

The worked example uses mean absolute error (MAE), where lower is better. BET computes a score for each sampled prediction and individual. A mean across posterior draws summarizes errors under the fitted model; draws are not independent patients or repeated validation experiments. In particular, the mean of MAEs computed on sampled trajectories need not equal the MAE of the posterior-mean trajectory.

A single overall mean can hide differences between patients. To check whether personalization helps broadly rather than only for a few individuals, retain the test-individual axis:

```
before_by_person = jnp.mean(before, axis=(0, 1))
after_by_person = jnp.mean(after, axis=(0, 1))
print("MAE without calibration, per person:", before_by_person)
print("MAE with calibration, per person:", after_by_person)
```

These arrays have one value per test patient. Quantiles across posterior draws describe uncertainty conditional on the fitted model; they are **not** confidence intervals for performance across future patient populations.

Check what uncertainty is represented

Using positive `nGlobalSamples` propagates uncertainty in learned global parameters and correlations; local z-score draws represent uncertainty and variation between individuals. Deterministic model outputs do not automatically include new observation noise, and none of these draws accounts for possible errors in the model's structure.

For stronger evidence of predictive performance, evaluate more held-out individuals, repeat across data splits or random seeds, and vary how many early observations are available for personalization. These checks address different questions from simply drawing more posterior samples.

Part 9: Troubleshooting

Bayesian inference can be fragile when a model is difficult to fit. Parameters that compensate for one another, parameters that barely affect predictions, and very large or unstable gradients can make reliable fitting difficult or impossible. Complex dynamics and likelihoods can make the problem harder still.

Build complexity incrementally. Start with a simple model that fits reliably, ideally checking parameter recovery on synthetic data with known values. Add one mechanism or parameter at a time and check that inference still behaves sensibly before proceeding. If fitting breaks down, investigate the most recent addition rather than debugging the entire complex model at once. Building toward a complex model is usually easier to debug than fitting it all at once.

Keep model evaluations fast and differentiable. NUTS and SVI may evaluate the model and its gradients many thousands of times during training or test-time personalization. For differential-equation models, use a differentiable ODE solver rather than manually simulating the dynamics with a Python time-step loop; use an analytical solution when one is available. Check runtime across plausible parameter values: regimes that force a numerical solver to take tiny steps can make inference prohibitively slow.

For specific failures, use the checks below. See Part 4 for the API reference.

Common problems

Shape mismatch. Start with one entry and print array shapes before tracing. All top-level data arrays must have the same entry count, and per-entry model outputs must match likelihood observation shapes. Broadcasting can silently produce an incorrect

likelihood, so verify dimensions explicitly.

Invalid entry indices. Training and test indices must be nonempty, one-dimensional collections of unique integers within the data's entry range. BET rejects duplicate, negative, and out-of-range indices; it does not silently remove duplicates or interpret negative indices as Python indexing from the end.

Local z-scores mistaken for physical parameters. A local z-score has a standard-normal marginal prior and is not measured in units of growth rate, drug sensitivity, or observation noise. Specify each transformation to physical units explicitly, and document the resulting units.

Hidden test outcomes leaking into inference. Keeping the full `y` array for scoring is safe only if the test likelihood masks unavailable outcomes and the mechanistic model does not read them. Check preprocessing, features, and masks to prevent future measurements from influencing predictions.

Missing data and NaNs. Masked observations still require finite values at points where JAX evaluates intermediate expressions. Use safe finite placeholders and keep ODE solver time grids valid even for padded observations.

Unexpectedly narrow intervals. With `nGlobalSamples=None`, BET fixes global parameters at posterior medians and does not propagate their uncertainty. Use a positive `nGlobalSamples` to include it. Approximate SVI posteriors may also underestimate uncertainty.

Poor convergence. Check observation scaling, priors, parameter identifiability, and solver gradients. For NUTS, inspect mixing and convergence diagnostics; for SVI, inspect the loss trajectory and sensitivity to initialization.

Save/Load mismatch. Restore compatible data shapes, Method configuration, model function, and global prior function. Python functions are not serialized; stored names are metadata, not executable code.

Advanced: Adding a custom inference method

Most users can rely on NUTS or SVI. This section explains how developers can add another inference algorithm without changing BET's training, testing, or scoring workflows.

BET separates inference algorithms from its population and individual workflows. NUTS and SVI are built in; to integrate another backend, implement the minimum Method interface and validate it before fitting.

A configured Method instance must implement:

- `_Fit(ModelFn, rngKey, samplePrefixes)`: run inference and return a dictionary with `"output"` (backend-specific results or diagnostics) and `"samples"` (posterior or explicitly approximate-posterior draws).
- `_SaveArgs()`: return a dictionary of constructor keyword arguments so BET can reconstruct the Method from saved configuration.

The `samples` dictionary maps NumPyro sample-site names to arrays with *draws along the first axis*. Retain only sites whose names start with one of `samplePrefixes`; BET uses these names to identify global parameters, local z-scores, Cholesky factors, and optionally model outputs.

This minimal skeleton illustrates the interface:

```
class MyMethod:
    _printMode = None
```

```
    def __init__(self, nSamples=100):
        self.nSamples = nSamples

    def _Fit(self, ModelFn, rngKey, samplePrefixes):
        # Replace with an actual inference backend.
        rawSamples, backendOutput = run_my_inference(ModelFn, rngKey, self.nSamples)
        samples = {name: value for name, value in rawSamples.items()
                   if any(name.startswith(prefix) for prefix in samplePrefixes)}
        return {"output": backendOutput, "samples": samples}

    def _SaveArgs(self):
        return {"nSamples": self.nSamples}
```

`run_my_inference` is a placeholder, *not a BET function*. The backend must produce meaningful posterior or clearly identified approximate-posterior draws. Repeating a maximum-a-posteriori (MAP) point does not measure posterior uncertainty.

Validate the Method's structure before using it:

```
import BayesianEstimationTool as bet

method = MyMethod(nSamples=100)
bet.ValidateMethod(method)
```

`ValidateMethod` checks required callable signatures, saved constructor arguments, and printing hooks. It *does not execute* `_Fit`, inspect returned samples or shapes, or establish statistical correctness. Test those separately on small synthetic models and with posterior predictive checks.

Optional features include:

- **Diagnostics and printing:** Set `_printMode` to `"before"`, `"after"`, or `"replace"`, and implement `_Print(output, localNames, globalNames, title=None)`. With `_printMode=None`, BET uses its generic sample summary.
- **Configuration:** Accept options for initialization, optimization, sampling, and progress reporting, and return reproducible constructor options from `_SaveArgs()`.
- **Persistence:** Keep configuration reconstructible through constructor keyword arguments. The backend `output` is returned to the caller but is not automatically serialized as a live inference-engine session.
- **Inference quality:** Compare posterior summaries, predictive distributions, and computational costs against NUTS on a known model. This is behavioral testing, not structural validation.

See `NutsMethod.py` and `SviMethod.py` for complete Method implementations.